NMPDE Projects: Comprehensive Testing and CI/CD Documentation

Advanced Methods for Scientific Computing

February 2026

Abstract

This document provides a comprehensive overview of the testing and continuous integration infrastructure developed for the NMPDE space-time adaptivity project. It covers the automated verification pipeline, scalability testing framework, model validation mechanisms, and Azure cloud deployment architecture. The documentation serves as a reference guide for developers and stakeholders involved in the project.

Contents

1 Introduction and Project Overview

1.1 Project Scope

The NMPDE (Numerical Methods for Partial Differential Equations) projects repository implements advanced numerical algorithms for solving partial differential equations, with a specific focus on space-time adaptivity techniques. The primary deliverable is the 'space-time-adaptivity' executable, built using the *deal.II* finite element library and implemented in C++.

1.2 Key Components

The project structure includes:

- **Source Code** (`src/`): Core implementation in C++, including the Heat equation solver and manufactured solution tests
- **Mesh Files** (`mesh/`): Gmsh-generated mesh files for various refinement levels
- **Build System**: CMake-based configuration using deal.II's integration
- **Testing Infrastructure** (`scripts/`): Bash and Python utilities for verification, validation, and performance analysis
- **CI/CD Pipelines** (`.github/workflows/`): GitHub Actions workflows for continuous integration
- **Infrastructure as Code** (`terraform/`): Terraform configurations for Azure cloud provisioning

1.3 Technology Stack

- **Language**: C++17 with deal.II library
- **Build System**: CMake (≥ 3.12)
- **Container Platform**: Docker (quay.io/pjbaioni/amsc_mk:2025)
- **CI/CD**: GitHub Actions
- **Cloud Infrastructure**: Microsoft Azure with Terraform
- **Testing Framework**: CTest (integrated with CMake)
- **Validation**: Python 3 with pyvista for VTU/PVTu mesh analysis

2 Testing Infrastructure Overview

2.1 Testing Philosophy

The project employs a multi-tiered testing approach:

1. **Unit Tests**: CTest-based tests defined in `CMakeLists.txt`
2. **Verification Tests**: Automated validation of compilation, execution, and correctness
3. **Model Validation Tests**: Computation-intensive tests verifying physical conservation laws
4. **Scalability Tests**: Performance benchmarking across varying thread counts
5. **Artifact Generation**: Automated production of visualization-ready output files (VTU/PVTu format)

2.2 Build Configuration

The project uses a standard CMake workflow:

```
cmake_minimum_required(VERSION 3.12.0)
project(space-time-adaptivity LANGUAGES CXX C)
include(common/cmake-common.cmake)

add_executable(space-time-adaptivity src/space-time-adaptivity.cpp src/
   Heat.cpp)
deal_ii_setup_target(space-time-adaptivity)

enable_testing()
add_test(NAME space-time-adaptivity COMMAND space-time-adaptivity)
add_test(NAME model_validation COMMAND space-time-adaptivity)
```

Key CMake options:

- `CMAKE_BUILD_TYPE=Release`: Optimized compilation for performance
- `DDEAL_II_ALLOW_TESTS=ON`: Enable deal.II test support

2.3 Local Testing Execution

2.3.1 Verification Testing

The verification script configures, builds, and runs all registered tests:

```
#!/bin/bash
./scripts/run_verification.sh [build_dir]
```

This script:

1. Configures the project with Release optimization
2. Builds all targets in parallel
3. Executes CTest with parallel execution on $(nproc) - 1 cores$
3. Reports test results with full failure diagnostics

2.3.2 Model Validation Testing

The model validation harness provides sophisticated verification capabilities:

```
TEST_CMD='./build/bin/project5_generate_results␣>␣model_results.csv' \
  RESULTS_FILE=model_results.csv TOL=1e-6 \
  PY_VALIDATOR=scripts/validate_model.py \
  ./scripts/run_model_validation.sh build
```

Environment variables for customization:

- `VENV_DIR`: Python virtual environment location (default: `venv`)
- `BUILD_DIR`: Build directory containing executables
- `RESULTS_FILE`: Output file containing numerical results
- `TEST_CMD`: Custom command to generate results
- `PY_VALIDATOR`: Path to Python validation script
- `TOL`: Relative error tolerance for validation (default: `1e-6`)

2.3.3 Scalability Testing

Thread-based scalability analysis with timing measurements:

```
TEST_EXEC=build/space-time-adaptivity ./scripts/run_scalability.sh
   build
```

Output format (CSV):

```
threads,seconds,cmd
1,45.32,build/space-time-adaptivity --threads=1
2,23.18,build/space-time-adaptivity --threads=2
4,12.45,build/space-time-adaptivity --threads=4
8,7.33,build/space-time-adaptivity --threads=8
16,4.92,build/space-time-adaptivity --threads=16
```

3 GitHub Actions CI/CD Pipeline

3.1 Architecture Overview

The GitHub Actions infrastructure consists of two coordinated workflows:

- **Verification CI**: Automatic on every push and pull request
- **Scalability Tests**: Manual dispatch workflow with optional executable parameter

Both workflows execute within the standardized Docker container `quay.io/pjbaioni/amsc_mk:2025`, ensuring reproducibility across deployment environments.

3.2 Verification Workflow

The verification workflow (`.github/workflows/verification.yml`) implements a two-stage pipeline:

3.2.1 Stage 1: Build and Test

- **Environment**: Ubuntu latest with containerized deal.II dependencies
- **Modules**: Loads `gcc-glibc` and `dealii` via environment modules
- **Build**: Parallel CMake configuration and compilation
- **Testing**: Executes CTest with parallel execution strategy
- **Artifacts**: Uploads entire build directory for downstream model validation

3.2.2 Stage 2: Model Validation

- **Dependency**: Requires successful build-and-test completion
- **Environment**: Isolated job with download of build artifacts
- **Execution**: Runs `scripts/run_model_validation.sh`
- **Validation**: Executes Python-based model correctness verification

Key Features:

- Automatic triggering on push and pull requests
- Parallel build optimization ($(nproc) - 1 threads$)
- Artifact retention for cross-job communication
- Comprehensive failure diagnostics with full output

3.3 Scalability Workflow

The scalability workflow (`.github/workflows/scalability.yml`) provides configuration flexibility:

- **Trigger**: Manual dispatch (`workflow_dispatch`)
- **Input Parameter**: `test_executable` (default: `build/space-time-adaptivity`)
- **Build Step**: Clean rebuild of project in Release mode

- **Execution**: Runs `scripts/run_scalability.sh` with provided executable
- **Output**: Generates `scalability_results.csv`

Workflow dispatch allows flexibility such as:

```
# Run with custom benchmark executable
TEST_EXEC=build/custom-benchmark ./scripts/run_scalability.sh build

# Or default to ctest scalability tests
./scripts/run_scalability.sh build
```

4 Model Validation Framework

4.1 Validation Architecture

The model validation system provides sophisticated verification for numerical solutions through multiple analysis pathways:

4.1.1 Input Format Support

1. **CSV Numeric Files**: Direct numerical result files
2. **VTU Files**: Individual time-step visualization files from deal.II's DataOut
3. **PVTu Files**: Master parallel visualization files (typically used for analysis)

4.1.2 Analysis Methodology

For VTU/PVTu format outputs, the validator:

1. Reads mesh data using PyVista library
2. Extracts the `solution` field from point or cell data
3. Computes volume-weighted mean (mass integral)
4. Compares against known reference value
5. Reports relative error and pass/fail status

4.2 Validation Script Implementation

The Python validator (`scripts/validate_model.py`) provides multiple validation strategies:

```
def compute_mean_from_vtu(path: Path):
    """Computes␣the␣mean␣of␣the␣'solution'␣data␣from␣VTU/PVTu."""
    import pyvista as pv
    mesh = pv.read(str(path))

    if 'solution' in mesh.point_data:
        return float(np.mean(mesh.point_data['solution']))
    if 'solution' in mesh.cell_data:
        return float(np.mean(mesh.cell_data['solution']))

    # Fallback to first available data array
    if mesh.point_data:
        key = list(mesh.point_data.keys())[0]
        return float(mesh.point_data[key][0])
    raise ValueError('No␣suitable␣data␣arrays␣found')
```

4.3 Physical Conservation Properties

The default validation target assumes the `FunctionU0` initial condition over the unit domain $[0, 1]^3$. The expected volume-weighted mean is:

$$\text{Expected Mass} = \int_\Omega u_0(x)\, dx = \frac{1}{216}$$

The numerical scheme with homogeneous Neumann boundary conditions should conserve this quantity. Validation succeeds when:

$$\left| \frac{\text{Computed Mean} - \frac{1}{216}}{\frac{1}{216}} \right| < \text{TOL}$$

where TOL is configurable (default: 10^{-6}).

5 Azure Cloud Infrastructure and Deployment

5.1 Infrastructure Overview

The Azure infrastructure provides scalable cloud resources for distributed testing and performance monitoring. Terraform manages all provisioned resources, ensuring reproducibility and version control.

5.2 Resource Architecture

5.2.1 Resource Group

Name: `nmpde-projects-rg`

- Central container for all resources
- Ensures logical grouping and lifecycle management
- Supports role-based access control (RBAC)

5.2.2 Container Registry (ACR)

Name: `nmpdeprojectsacr`

- Tier: Basic (suitable for development/testing)
- Admin access enabled for GitHub Actions authentication
- Mirrors container images (e.g., quay.io/pjbaioni/amsc_mk:2025)
- Integration point between GitHub Actions and Azure deployment

5.2.3 Storage Account

Name: `nmpdeprojectsstorage`

- Tier: Standard with Local Redundancy (LRS)
- Blob containers for organizational separation:
  1. `performance-reports`: JSON and CSV test outputs
  2. `build-artifacts`: Compiled binaries and libraries
  3. `test-results`: CTest and benchmark results
- Access: Private (requires authentication)

5.2.4 Log Analytics Workspace

Configuration: PerGB2018 pricing tier

- Centralizes monitoring and diagnostic data
- Enables performance analysis and troubleshooting
- Integrates with Azure Monitor for alerting

5.3 Infrastructure as Code (Terraform)

5.3.1 Provider Configuration

```
terraform {
  required_version = ">= 1.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

5.3.2 Deployment Process

1. **Initialization**: `terraform init` (downloads providers)
2. **Planning**: `terraform plan` (previews changes)
3. **Application**: `terraform apply` (provisions resources)
4. **Cleanup**: `terraform destroy` (removes resources)

5.3.3 Variable Configuration

Key Terraform variables:

- `subscription_id`: Azure subscription identifier
- `resource_group_name`: Logical container name
- `location`: Azure region (e.g., eastus)
- `acr_name`: Container registry identifier
- `storage_account_name`: Storage account identifier

5.4 GitHub Actions Integration

5.4.1 Service Principal Setup

Azure resources require GitHub Actions to authenticate. Setup process:

```
# Create service principal with Contributor role
az ad sp create-for-rbac \
  --name "github-actions-nmpde" \
  --role contributor \
  --scopes /subscriptions/5d7fbbd4-db18-4472-915d-de671cc0f22a \
  --sdk-auth
```

Output JSON contains credentials stored as GitHub secret `AZURE_CREDENTIALS`.

5.4.2 Secrets Configuration

Repository secrets required:

- `AZURE_CREDENTIALS`: Service principal authentication JSON
- `AZURE_SUBSCRIPTION_ID`: Subscription identifier
- `AZURE_RESOURCE_GROUP`: Resource group name

5.5 Performance Monitoring and Reporting

The performance testing script (`scripts/run_performance_tests.sh`) collects comprehensive system metrics:

5.5.1 Collected Metrics

- **CPU Information**: Core count, architecture, flags (from `lscpu`)
- **Memory**: Total, available, and used capacity (from `free`)
- **Disk**: Storage capacity and utilization (from `df`)
- **Process Limits**: Maximum processes and memory constraints (from `ulimit`)
- **Timing Data**: Execution duration and throughput metrics

5.5.2 Output Format

Reports are generated in two formats:

1. **JSON Report** (`performance_YYYYMMDD_HHMMSS.json`): Structured data for automated processing
2. **Text Report** (`system_metrics_YYYYMMDD_HHMMSS.txt`): Human-readable system information

Reports are timestamped to enable historical trend analysis and performance regression detection.